

Map-STDP

Vikram Ramavarapu¹

¹Computer Science, University of Illinois, Urbana, Illinois, USA.

Contributing authors: vikramr2@illinois.edu;

Abstract

STDP is a biologically plausible learning rule that has been widely used in spiking neural networks. However, STDP is a local learning rule that cannot capture the global modular structure of the neural network. In this paper, a new Map-STDP rule is derived using the map equation, which is an info-theoretic objective function for community detection.

1 Introduction

Spike-timing dependent plasticity (STDP) [1] is a biologically plausible learning rule that has been widely used in spiking neural networks. When a pre-synaptic neuron fires shortly before a post-synaptic neuron, the synapse between them is strengthened. Conversely, if the pre-synaptic neuron fires shortly after the post-synaptic neuron, the synapse is weakened.

However, STDP is a fully local rule that cannot capture the global structure of the neural network. This issue is much more prominent when population vectors are used for inference via the population of neurons in different regions of the network. In order to ensure the expressivity of the population vectors, the neural network must have a modular structure, much like the cortical organization of the brain.

Moreover, neural dynamics are very stochastic and operate as a form of random walk [2], which is a major component of the map equation, an info-theoretic objective function for community detection in the Infomap algorithm [3]. The map equation measures the modular structure of a network by quantifying the amount of information needed to describe the flow of activity through the network via a random walk.

As such, a Map-STDP rule that incorporates the map equation is derived. The Map-STDP rule is a local learning rule that can capture the global modular structure of the neural network. Moreover, the Map-STDP rule preserves biorealism through

the use of info-theoretic entropy maximization, stochastic neural dynamics, and local synaptic updates.

2 Infomap and Entropy

2.1 Infomap

The Infomap algorithm is a community detection algorithm that minimizes the map equation to identify communities in a network. The map equation is defined as:

$$L(M) = q_{\cap} H(\mathcal{Q}) + \sum_{m \in M} p_{\cap}^m H(\mathcal{P}^m)$$

where M is a partition of the network into m communities. There are two terms. The first term represents the entropy of movement between communities, while the second term represents the entropy of movement within communities, where exiting the community is also considered a movement.

2.2 Neurological Entropy

The brain also operates with two different types of entropy. The first is response entropy of the individual neurons [4]. Maximizing this entropy ensures that the neurons are utilizing their full dynamic range, so that they are as expressive as possible. Formally, for a neuron with firing rate r , and maximum firing rate r_{max} , the response entropy is defined as:

$$H = \log_2 \left(\frac{r_{max}}{\Delta r} \right)$$

assuming a uniform distribution of firing rates, where Δr is the resolution of the firing rate. With this entropy value emerges mutual information between the stimulus and the neural response, which is a measure of how much information the neural response conveys about the stimulus. This is formalized as:

$$I(r; a) = H - \langle H_a \rangle_a$$

The second is thermodynamic entropy, otherwise known as free energy [5]. Given p , the brain's probabilistic model of the world, and q , the brain's recognition density, the free energy is defined as:

$$F = \mathbb{E}_q[-\log(z, o)] - H(q(z))$$

Where z is the latent variables and o is the sensory input. The first term is the expected negative log likelihood of the brain's model of the world, which is also known as the energy. The second term is the entropy of the brain's recognition density. This can be expanded further as:

$$H(q) = \mathbb{E}_q[-\log q(z)]$$

Therefore,

$$F = \mathbb{E}_q[\log q(z) - \log p(z, o)] = D_{KL}(q(z) || p(z, o)) - \log p(o)$$

2.3 Connecting the Map Equation to Neurological Entropy

The map equation can be connected to both neurological entropies mentioned above through the theme of surprise minimization. Response entropy minimizes surprise at the neural level, while free energy minimizes surprise at the network level. The map equation addresses the meso-scale level.

First, with response entropy, standard STDP already maximizes mutual information between the stimulus and the neural response. This means that the Map-STDP rule would be optimally utilized when compounded with an STDP rule that maximizes response entropy, while the map equation term acts as a modular gating mechanism.

Second, with free energy, the map equation can be seen as operating on the same underlying distribution that free energy minimization converges to. Specifically, the stationary distribution that emerges from the SNN's MCMC dynamics, $p(\mathbf{z}|\mathbf{o})$ [2], is precisely the distribution that minimizes variational free energy in Equation ??, that is, the optimal recognition density $q(\mathbf{z}) = p(\mathbf{z}|\mathbf{o})$ at the free energy minimum. The map equation operates on the marginal visit frequencies $\tilde{p}_i = p(z_i = 1|\mathbf{o})$ that emerge as a byproduct of this same sampling process. In this sense the map equation and free energy minimization are complementary: free energy minimization defines what the network should represent, while the map equation defines how the mesoscale structure of the network should be organized to represent it efficiently.

3 Deriving Map-STDP

As mentioned earlier, in order to properly address response entropy, the Map-STSP rule should be compounded with an STDP rule that maximizes response entropy. As such, we can start with a rule in the form of:

$$\Delta W_{ij} = \eta \cdot \text{STDP}(i, j) - \eta' \cdot G(i, j)$$

where $\text{STDP}(i, j)$ is a standard STDP rule that maximizes response entropy, $G(i, j) = \partial L / \partial W_{ij}$ is the gradient of the map equation, and η, η' are separate learning rates for the two objectives. The two terms are additive rather than multiplicative, so each objective acts independently: STDP drives the synaptic updates toward maximizing response entropy, while the map gradient term descends the map equation loss to encourage modular structure. The goal is to derive the form of $G(i, j)$.

In its general form, a random walk on a network has a transition matrix T that can be defined as:

$$T_{ij} = \frac{W_{ij}}{\sum_k W_{ik}}$$

where W_{ij} is the weight of the edge from node i to node j . We can then tie this in to Buesing et. al.'s equation for the membrane potential of a neuron in an SNN that performs MCMC sampling [2]:

$$u_i = b_i + \sum_j W_{ij} z_j(t)$$

where u_i is the membrane potential of neuron i , b_i is the bias of neuron i , and $z_j(t)$ is the binary spike variable of neuron j at time t . While z is an instantaneous variable, we can take its expectation over time in order to make it compatible with the map equation's flow-based framework. This gives us:

$$\langle z_j \rangle = \tilde{p}_j = p(z_j = 1 | \mathbf{o})$$

This gives us the corrected definition of our transition matrix as:

$$\tilde{T}_{ij} = \frac{W_{ij}\tilde{p}_j}{\sum_k W_{ik}\tilde{p}_k} = \frac{\tilde{W}_{ij}}{\tilde{s}_i}$$

With our new defined variables $\tilde{W}_{ij} = W_{ij}\tilde{p}_j$ and $\tilde{s}_i = \sum_k W_{ik}\tilde{p}_k$.

The Chapman-Kolmogorov equation then gives us the probability of being at node j at time t as:

$$\pi_j^{t+1} = \sum_i \tilde{T}_{ij} \pi_i^t$$

The stationary distribution therefore satisfies:

$$\tilde{p}_j = \sum_i \tilde{T}_{ij} \tilde{p}_i \rightarrow \tilde{p} = \tilde{T} \tilde{p}$$

We can rule out the trivial solution of $\tilde{p} = 0$ by enforcing a normalization constraint of $\sum_j \tilde{p}_j = 1$. Another trivial solution we can rule out is the one where $\tilde{T} = I$, which could only be true if we don't add a constraint of irreducibility, meaning there must be some level of connectivity between all nodes in the network. This is then reducible to a fixed point problem, therefore we can then solve for $\tilde{p}$ using power iteration:

$$\tilde{p}^{(k+1)} = \tilde{T} \tilde{p}^{(k)}$$

Which can be estimated online via the local update rule:

$$\tilde{p}_i \leftarrow (1 - \beta) \tilde{p}_i + \beta \sum_j \tilde{T}_{ij} z_j(t) \quad (1)$$

It is worth noting that the stationary probability $\tilde{p}$ is also the PageRank of the network with transition matrix $\tilde{T}$. The above rule is actually a form of the iterative step of the power iteration method for computing PageRank.

The proofs of convergence are deferred to Appendix A.

Now, it's time to come back to the map equation and derive the form of $G(i, j)$. We can rewrite $p_{\odot}^m$ and $q_{\sim}$ in terms of $\tilde{p}$:

$$\begin{aligned} p_m &:= \sum_{i \in m} \tilde{p}_i \\ q_m &:= \sum_{i \in m} \sum_{j \notin m} \tilde{T}_{ij} \tilde{p}_i \\ p_{\odot}^m &= p_m + q_m \end{aligned}$$

$$q_{\curvearrowright} = \sum_{m \in M} q_m \quad (2)$$

Where p_m is the total visit frequency of community m , and q_m is the total exit frequency of community m . We can then rewrite the map equation as:

$$L(M) = q_{\curvearrowright} H(\mathcal{Q}) + \sum_{m \in M} (p_m + q_m) H(\mathcal{P}^m)$$

Now we need to expand the entropy terms.

$$H(\mathcal{Q}) = - \sum_{m \in M} \frac{q_m}{q_{\curvearrowright}} \log \left(\frac{q_m}{q_{\curvearrowright}} \right)$$

$$H(\mathcal{P}^m) = - \frac{q_m}{p_m + q_m} \log \left(\frac{q_m}{p_m + q_m} \right) - \sum_{i \in m} \frac{\tilde{p}_i}{p_m + q_m} \log \left(\frac{\tilde{p}_i}{p_m + q_m} \right)$$

Now, substituting this back into the map equation gives us:

$$\begin{aligned} L(M) &= - \sum_{m \in M} q_m \log \left(\frac{q_m}{q_{\curvearrowright}} \right) \\ &\quad - \sum_{m \in M} (p_m + q_m) \left[- \frac{q_m}{p_m + q_m} \log \left(\frac{q_m}{p_m + q_m} \right) - \sum_{i \in m} \frac{\tilde{p}_i}{p_m + q_m} \log \left(\frac{\tilde{p}_i}{p_m + q_m} \right) \right] \\ &= - \sum_{m \in M} q_m \log \left(\frac{q_m}{q_{\curvearrowright}} \right) + \sum_{m \in M} q_m \log \left(\frac{q_m}{p_m + q_m} \right) + \sum_{m \in M} \sum_{i \in m} \tilde{p}_i \log \left(\frac{\tilde{p}_i}{p_m + q_m} \right) \end{aligned}$$

Now we can take the gradient with respect to W_{ij} . The first term requires two dependent derivatives via the chain rule: $\partial q_m / \partial W_{ij}$ and $\partial q_{\curvearrowright} / \partial W_{ij}$. The second term also requires two dependent derivatives: $\partial q_m / \partial W_{ij}$ and $\partial p_m / \partial W_{ij}$. The third term requires two dependent derivatives: $\partial \tilde{p}_i / \partial W_{ij}$ and $\partial p_m / \partial W_{ij}$.

Starting with computing $\partial \tilde{p}_i / \partial W_{ij}$:

$$\begin{aligned} \frac{\partial \tilde{p}_i}{\partial W_{ij}} &= \left[\frac{\partial \tilde{T} \tilde{p}}{\partial W_{ij}} \right]_i \\ &= \left[\frac{\partial \tilde{T}}{\partial W_{ij}} \tilde{p} + \tilde{T} \frac{\partial \tilde{p}}{\partial W_{ij}} \right]_i = \left[\frac{\partial \tilde{p}}{\partial W_{ij}} \right]_i \end{aligned}$$

Removing the $[\cdot]_i$ operator, this gives us a fixed point partial differential equation. We can solve this as follows:

$$\frac{\partial \tilde{p}}{\partial W_{ij}} = (I - \tilde{T})^{-1} \frac{\partial \tilde{T}}{\partial W_{ij}} \tilde{p}$$

This raises the issue of inverting a global matrix in order to compute a local synaptic update. We can sidestep this issue by using the local update rule in Equation 1 to estimate $\tilde{p}$ online and then treating $\tilde{p}$ as a constant with respect to W_{ij} when computing the gradient.

Similarly, $\partial p_m / \partial W_{ij}$ can be computed as:

$$\frac{\partial p_m}{\partial W_{ij}} = \sum_{i' \in m} \frac{\partial \tilde{p}_{i'}}{\partial W_{ij}}$$

This raises the same issue of a global matrix inversion. Instead, we maintain a scalar running estimate $g_{ij}(t) \approx \partial \tilde{p}_i / \partial W_{ij}$ per synapse, obtained by differentiating the EMA update in Equation 1 with respect to W_{ij} . Since $\partial \tilde{T}_{kk'} / \partial W_{ij} = 0$ for all $k \neq i$, the full gradient vector $\partial \tilde{p} / \partial W_{ij}$ has only one nontrivial component at index $k = i$. Defining $g_{ij}(t) := \partial \tilde{p}_i / \partial W_{ij}$, the update rule is:

$$g_{ij}(t+1) = (1 - \beta) g_{ij}(t) + \beta \sum_{k'} \frac{\partial \tilde{T}_{ik'}}{\partial W_{ij}} z_{k'}(t) \quad (3)$$

This requires only one scalar per synapse, so storage is $O(N^2)$, the same as W . The convergence of this estimator is proved in Appendix A. With this estimate in hand, $\partial p_m / \partial W_{ij}$ collapses to:

$$\frac{\partial p_m}{\partial W_{ij}} \approx g_{ij}(t) \cdot \mathbf{1}[i \in m]$$

since $g_{ij,k} = 0$ for all $k \neq i$.

Now let's rewrite the map equation as follows:

$$L(M) = A + B + C$$

Where:

$$\begin{aligned} A &:= - \sum_{m \in M} q_m \log \left(\frac{q_m}{q_{\curvearrowright}} \right) \\ B &:= \sum_{m \in M} q_m \log \left(\frac{q_m}{p_m + q_m} \right) \\ C &:= \sum_{m \in M} \sum_{i \in m} \tilde{p}_i \log \left(\frac{\tilde{p}_i}{p_m + q_m} \right) \end{aligned}$$

Then, the gradient can be computed as:

$$\frac{\partial L}{\partial W_{ij}} = \frac{\partial A}{\partial W_{ij}} + \frac{\partial B}{\partial W_{ij}} + \frac{\partial C}{\partial W_{ij}}$$

Starting with $\partial A / \partial W_{ij}$:

$$\frac{\partial A}{\partial W_{ij}} = - \sum_{m \in M} \left[\frac{\partial q_m}{\partial W_{ij}} \log \left(\frac{q_m}{q_{\curvearrowright}} \right) + q_m \frac{\partial}{\partial W_{ij}} \log \left(\frac{q_m}{q_{\curvearrowright}} \right) \right]$$

$$\begin{aligned}
&= - \sum_{m \in M} \left[\frac{\partial q_m}{\partial W_{ij}} \log \left(\frac{q_m}{q_{\curvearrowright}} \right) + q_m \left(\frac{1}{q_m} \frac{\partial q_m}{\partial W_{ij}} - \frac{1}{q_{\curvearrowright}} \frac{\partial q_{\curvearrowright}}{\partial W_{ij}} \right) \right] \\
&= - \sum_{m \in M} \left[\frac{\partial q_m}{\partial W_{ij}} \log \left(\frac{q_m}{q_{\curvearrowright}} \right) + \frac{\partial q_m}{\partial W_{ij}} - \frac{q_m}{q_{\curvearrowright}} \frac{\partial q_{\curvearrowright}}{\partial W_{ij}} \right]
\end{aligned}$$

Now for $\partial B / \partial W_{ij}$:

$$\begin{aligned}
\frac{\partial B}{\partial W_{ij}} &= \sum_{m \in M} \left[\frac{\partial q_m}{\partial W_{ij}} \log \left(\frac{q_m}{p_m + q_m} \right) + q_m \frac{\partial}{\partial W_{ij}} \log \left(\frac{q_m}{p_m + q_m} \right) \right] \\
&= \sum_{m \in M} \left[\frac{\partial q_m}{\partial W_{ij}} \log \left(\frac{q_m}{p_m + q_m} \right) + q_m \left(\frac{1}{q_m} \frac{\partial q_m}{\partial W_{ij}} - \frac{1}{p_m + q_m} \left(\frac{\partial p_m}{\partial W_{ij}} + \frac{\partial q_m}{\partial W_{ij}} \right) \right) \right] \\
&= \sum_{m \in M} \left[\frac{\partial q_m}{\partial W_{ij}} \log \left(\frac{q_m}{p_m + q_m} \right) + \frac{\partial q_m}{\partial W_{ij}} - \frac{q_m}{p_m + q_m} \left(\frac{\partial p_m}{\partial W_{ij}} + \frac{\partial q_m}{\partial W_{ij}} \right) \right]
\end{aligned}$$

And finally for $\partial C / \partial W_{ij}$:

$$\begin{aligned}
\frac{\partial C}{\partial W_{ij}} &= \sum_{m \in M} \sum_{k \in m} \left[\frac{\partial \tilde{p}_k}{\partial W_{ij}} \log \left(\frac{\tilde{p}_k}{p_m + q_m} \right) + \tilde{p}_k \frac{\partial}{\partial W_{ij}} \log \left(\frac{\tilde{p}_k}{p_m + q_m} \right) \right] \\
&= \sum_{m \in M} \sum_{k \in m} \left[\frac{\partial \tilde{p}_k}{\partial W_{ij}} \log \left(\frac{\tilde{p}_k}{p_m + q_m} \right) + \tilde{p}_k \left(\frac{1}{\tilde{p}_k} \frac{\partial \tilde{p}_k}{\partial W_{ij}} - \frac{1}{p_m + q_m} \left(\frac{\partial p_m}{\partial W_{ij}} + \frac{\partial q_m}{\partial W_{ij}} \right) \right) \right] \\
&= \sum_{m \in M} \sum_{k \in m} \left[\frac{\partial \tilde{p}_k}{\partial W_{ij}} \log \left(\frac{\tilde{p}_k}{p_m + q_m} \right) + \frac{\partial \tilde{p}_k}{\partial W_{ij}} - \frac{\tilde{p}_k}{p_m + q_m} \left(\frac{\partial p_m}{\partial W_{ij}} + \frac{\partial q_m}{\partial W_{ij}} \right) \right]
\end{aligned}$$

Since we are treating $\tilde{p}$ as a constant with respect to W_{ij} , the $\partial \tilde{p}_k / \partial W_{ij}$ terms can be removed from the gradient, which gives us the final form of the gradient as:

$$\frac{\partial C}{\partial W_{ij}} = \sum_{m \in M} \sum_{k \in m} \left[- \frac{\tilde{p}_k}{p_m + q_m} \left(\frac{\partial p_m}{\partial W_{ij}} + \frac{\partial q_m}{\partial W_{ij}} \right) \right]$$

Similarly, we can adjust all terms and remove all $\partial \tilde{p}_i / \partial W_{ij}$ terms, which gives us the following:

$$\begin{aligned}
\frac{\partial B}{\partial W_{ij}} &= \sum_{m \in M} \left[\frac{\partial q_m}{\partial W_{ij}} \log \left(\frac{q_m}{p_m + q_m} \right) + \frac{\partial q_m}{\partial W_{ij}} - \frac{q_m}{p_m + q_m} \left(\frac{\partial q_m}{\partial W_{ij}} \right) \right] \\
&= \sum_{m \in M} \frac{\partial q_m}{\partial W_{ij}} \left[\log \left(\frac{q_m}{p_m + q_m} \right) + 1 - \frac{q_m}{p_m + q_m} \right] \\
\frac{\partial C}{\partial W_{ij}} &= \sum_{m \in M} \sum_{k \in m} \left[- \frac{\tilde{p}_k}{p_m + q_m} \left(\frac{\partial q_m}{\partial W_{ij}} \right) \right]
\end{aligned}$$

$$= - \sum_{m \in M} \frac{\partial q_m}{\partial W_{ij}} \left[\sum_{k \in m} \frac{\tilde{p}_k}{p_m + q_m} \right]$$

Now we can compute $\partial q_m / \partial W_{ij}$. Recall that:

$$q_m = \sum_{i' \in m} \sum_{j' \notin m} \tilde{T}_{i'j'} \tilde{p}_{i'}$$

Since W_{ij} only appears in row i of $\tilde{T}$, the derivative is zero whenever $i \notin m$. For $i \in m$, we differentiate $\tilde{T}_{ij'} = \tilde{W}_{ij'} / \tilde{s}_i$ where $\tilde{s}_i = \sum_k W_{ik} \tilde{p}_k$.

Case 1: $i \notin m$. No dependence on W_{ij} :

$$\frac{\partial q_m}{\partial W_{ij}} = 0$$

Case 2: $i \in m, j \in m$. The numerator $\tilde{W}_{ij'}$ for exit edges $j' \notin m$ is unaffected, but the denominator $\tilde{s}_i$ gains $\tilde{p}_j$. So for each exit edge $j' \notin m$:

$$\frac{\partial \tilde{T}_{ij'}}{\partial W_{ij}} = - \frac{\tilde{T}_{ij'} \tilde{p}_j}{\tilde{s}_i}$$

Summing over all exit edges from i :

$$\frac{\partial q_m}{\partial W_{ij}} = - \frac{\tilde{p}_j \tilde{p}_i}{\tilde{s}_i} \sum_{j' \notin m} \tilde{T}_{ij'} = - \frac{\tilde{p}_j \tilde{p}_i e_i}{\tilde{s}_i}$$

where $e_i := \sum_{j' \notin m} \tilde{T}_{ij'}$ is the total exit probability of node i .

Case 3: $i \in m, j \notin m$. Now W_{ij} appears in both the numerator of $\tilde{T}_{ij}$ (an exit edge) and in the denominator $\tilde{s}_i$ of all exit edges. For the edge $j' = j$:

$$\frac{\partial \tilde{T}_{ij}}{\partial W_{ij}} = \frac{\tilde{p}_j \tilde{s}_i - \tilde{W}_{ij} \tilde{p}_j}{\tilde{s}_i^2} = \frac{\tilde{p}_j}{\tilde{s}_i} (1 - \tilde{T}_{ij})$$

For all other exit edges $j' \notin m, j' \neq j$, only the denominator is affected:

$$\frac{\partial \tilde{T}_{ij'}}{\partial W_{ij}} = - \frac{\tilde{T}_{ij'} \tilde{p}_j}{\tilde{s}_i}$$

Summing over all exit edges:

$$\frac{\partial q_m}{\partial W_{ij}} = \frac{\tilde{p}_i \tilde{p}_j}{\tilde{s}_i} \left[(1 - \tilde{T}_{ij}) - \sum_{\substack{j' \notin m \\ j' \neq j}} \tilde{T}_{ij'} \right] = \frac{\tilde{p}_i \tilde{p}_j}{\tilde{s}_i} (1 - e_i)$$

In summary:

$$\frac{\partial q_{m(i)}}{\partial W_{ij}} = \begin{cases} 0 & i \notin m \\ -\frac{\tilde{p}_j \tilde{p}_i e_i}{\tilde{s}_i} & i \in m, j \in m \\ \frac{\tilde{p}_j \tilde{p}_i}{\tilde{s}_i} (1 - e_i) & i \in m, j \notin m \end{cases} \quad (4)$$

The intuition is clear: adding weight to an intra-community edge ($j \in m$) rescales all exit probabilities downward, reducing q_m ; adding weight to a cross-community edge ($j \notin m$) increases q_m by the remaining intra-community probability mass ($1 - e_i$).

4 Training

Now that we have derived the Map-STDP rule, we can train an SNN using this rule. We however have multiple update steps that are interdependent: the synaptic updates depend on the online estimate of $\tilde{p}$, which in turn depends on the community assignments of the nodes. To address this, we use an alternating optimization approach between $\tilde{p}$ and W_{ij} , and every T steps we can reassign communities, and repeat this process until convergence.

Furthermore, although the Map-STDP rule was derived from Infomap’s map equation, it can be used with any community detection algorithm, as it only requires a community assignment M regardless of how it was generated. If the application requires a fixed set of outputs (e.g., classification), a clustering algorithm with a fixed number of communities can be used. However, for applications involving unsupervised representation learning, such as an SNN processing natural visual or auditory stimuli where the number of meaningful concepts is unknown and may change over time, a variable- k method is more appropriate. In such settings, a fixed number of communities would either merge concepts that should be distinct or artificially split those that should not be, whereas a variable- k algorithm like Leiden or Louvain naturally grows the number of communities as the weight structure becomes more differentiated, allowing the modular structure to reflect the true complexity of the input distribution. In either case, the community assignments are updated every T steps as described above.

The starting network can be initialized using a synthetic network generator. The type of network generator can be chosen based on the application. For example, if we require a constant number of communities, we can use the stochastic block model. If we require a variable number of communities, we can use the LFR benchmark. The network can also be initialized with random weights, which will then be updated using the Map-STDP rule.

References

- [1] Shouval, H.Z., Wang, S.S.-H., Wittenberg, G.M.: Spike timing dependent plasticity: a consequence of more fundamental learning rules. *Front. Comput. Neurosci.* **4** (2010)

- [2] Buesing, L., Bill, J., Nessler, B., Maass, W.: Neural dynamics as sampling: a model for stochastic computation in recurrent networks of spiking neurons. *PLoS Comput. Biol.* **7**(11), 1002211 (2011)
- [3] Rosvall, M., Bergstrom, C.T.: Maps of random walks on complex networks reveal community structure. *Proc. Natl. Acad. Sci. U. S. A.* **105**(4), 1118–1123 (2008)
- [4] Dayan, P., Abbott, L.F.: Information theory. In: *Theoretical Neuroscience: Computational and Mathematical Modeling of Neural Systems*, pp. 123–150. MIT Press, Cambridge, MA (2001). <https://doi.org/10.7551/mitpress/6755.001.0001>
- [5] Friston, K.: The free-energy principle: a unified brain theory? *Nat. Rev. Neurosci.* **11**(2), 127–138 (2010)

Appendix A Proofs and Remarks

Theorem 1 *The local update rule in Equation 1 converges to the stationary distribution of the random walk on the network.*

Proof Taking the expectation of the second term in the local update rule:

$$\mathbb{E} \left[\sum_j \tilde{T}_{ij} z_j(t) \right] = \sum_j \tilde{T}_{ij} \langle z_j \rangle = \sum_j \tilde{T}_{ij} \tilde{p}_j = [\tilde{T} \tilde{p}]_i$$

Substituting back into the local update rule gives:

$$\tilde{p}_i \leftarrow (1 - \beta) \tilde{p}_i + \beta [\tilde{T} \tilde{p}]_i$$

By the fixed point property of the stationary distribution $\tilde{p} = \tilde{T} \tilde{p}$, this reduces to:

$$\tilde{p}_i \leftarrow (1 - \beta) \tilde{p}_i + \beta \tilde{p}_i = \tilde{p}_i$$

so $\tilde{p}$ is a fixed point of the update rule. □

Theorem 2 *For fixed W , the online estimate $\tilde{p}_k(t)$ is an asymptotically unbiased approximator for $\langle z_k \rangle = p(z_k = 1 \mid \mathbf{o})$, i.e., $\lim_{t \rightarrow \infty} \mathbb{E}[\tilde{p}_k(t)] = \langle z_k \rangle$.*

Proof Unrolling the recursion in Equation 1 gives:

$$\mathbb{E}[\tilde{p}_i(t)] = (1 - \beta)^t \tilde{p}_i(0) + \beta \sum_{s=0}^{t-1} (1 - \beta)^{t-1-s} \mathbb{E} \left[\sum_j \tilde{T}_{ij} z_j(s) \right]$$

Since W is held fixed, $\mathbb{E}[\sum_j \tilde{T}_{ij} z_j(s)] = \sum_j \tilde{T}_{ij} \langle z_j \rangle = \tilde{p}_i$ by Theorem 1. Substituting:

$$\mathbb{E}[\tilde{p}_i(t)] = (1 - \beta)^t \tilde{p}_i(0) + \tilde{p}_i \cdot \beta \sum_{s=0}^{t-1} (1 - \beta)^{t-1-s}$$

Evaluating the geometric sum $\beta \sum_{s=0}^{t-1} (1-\beta)^{t-1-s} = 1 - (1-\beta)^t$:

$$\mathbb{E}[\tilde{p}_i(t)] = (1-\beta)^t \tilde{p}_i(0) + \tilde{p}_i \left(1 - (1-\beta)^t \right)$$

Since $0 < \beta \leq 1$, we have $(1-\beta)^t \rightarrow 0$ as $t \rightarrow \infty$, and therefore:

$$\lim_{t \rightarrow \infty} \mathbb{E}[\tilde{p}_i(t)] = \tilde{p}_i = \langle z_i \rangle$$

where the last equality follows from Buesing et al. [2], who establish that the stationary visit frequency of the random walk coincides with the marginal activation probability $p(z_i = 1 \mid \mathbf{o})$. $\square$

Theorem 3 For fixed W and fixed $\tilde{T}$, the scalar online gradient tracking estimate $g_{ij}(t)$ defined in Equation 3 satisfies $\lim_{t \rightarrow \infty} \mathbb{E}[g_{ij}(t)] = \partial \tilde{p}_i / \partial W_{ij}$.

Proof Since $\partial \tilde{T}_{kk'} / \partial W_{ij} = 0$ for all $k \neq i$, the fixed-point equation $\mathbf{v} = (I - \tilde{T})^{-1} \frac{\partial \tilde{T}}{\partial W_{ij}} \tilde{p}$ has $v_k = 0$ for $k \neq i$. It therefore suffices to track only the scalar $v_i = \partial \tilde{p}_i / \partial W_{ij}$.

Define $d := \sum_{k'} \frac{\partial \tilde{T}_{ik'}}{\partial W_{ij}} \tilde{p}_{k'}$, the unique nonzero entry of $\frac{\partial \tilde{T}}{\partial W_{ij}} \tilde{p}$. The scalar fixed-point equation for v_i is:

$$v_i = [\tilde{T} \mathbf{v}]_i + d = \sum_{k'} \tilde{T}_{ik'} v_{k'} + d$$

Since $v_{k'} = 0$ for $k' \neq i$, this reduces to $v_i = \tilde{T}_{ii} v_i + d$, giving $v_i = d / (1 - \tilde{T}_{ii})$.

Taking expectations of Equation 3 and using $\mathbb{E}[z_{k'}(t)] = \tilde{p}_{k'}$:

$$\mathbb{E}[g_{ij}(t+1)] = (1-\beta) \mathbb{E}[g_{ij}(t)] + \beta d$$

Unrolling this recursion:

$$\mathbb{E}[g_{ij}(t)] = (1-\beta)^t g_{ij}(0) + d \left(1 - (1-\beta)^t \right)$$

Since $0 < \beta \leq 1$, $(1-\beta)^t \rightarrow 0$ as $t \rightarrow \infty$, so $\mathbb{E}[g_{ij}(t)] \rightarrow d$.

Finally, note that the EMA learning rate β drives g_{ij} to d , not to $v_i = d / (1 - \tilde{T}_{ii})$. The discrepancy factor $(1 - \tilde{T}_{ii})$ is the probability that node i exits itself in one step, which is absorbed into the overall learning rate η' in the synaptic update rule. Therefore $g_{ij}(t)$ is an asymptotically unbiased estimator of $\partial \tilde{p}_i / \partial W_{ij}$ up to this constant rescaling. $\square$